

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Title	Visualizing Clusters with scikit-learn and Plotly			
Problem Statement	Perform Principal Component Analysis (PCA) on the Iris dataset and visualize the first two principal components using an interactive 2D scatter plot in Plotly.			
Objectives	• Load and verify the Iris dataset using Pandas and scikit-learn. • Standardize the four measurements before PCA. • Reduce the standardized data to two principal components. • Interpret explained variance and principal-axis coefficients. • Visualize species in an interactive Plotly scatter plot and save the results.			
Dataset URL Link / File	Built-in Iris dataset: sklearn.datasets.load_iris(as_frame=True). 150 samples; four numeric features measured in cm: sepal length, sepal width, petal length and petal width. Species: setosa (50), versicolor (50), virginica (50). Species labels are used for visualization/evaluation, not as input features. Dataset URL link:			
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads 150 Iris samples and species labels.
	2	DataFrame()	Pandas	Creates tables of components and coefficients.
	3	StandardScaler()	sklearn	Defines mean-zero, unit-variance scaling.
	4	fit_transform()	sklearn	Fits a transformer and transforms the data.
	5	PCA()	sklearn	Defines principal component analysis.
	6	concat()	Pandas	Combines components and original features.
	7	scatter()	Plotly	Creates an interactive 2D scatter plot.
	8	bar()	Plotly	Creates the explained-variance bar chart.
	9	groupby()	Pandas	Groups principal-component scores by species.
	10	mean()/std()	Pandas	Summarizes species centroids and spread.
	11	write_html()/write_image()	Plotly	Exports interactive HTML and static charts.
	12	to_csv()	Pandas	Exports the PCA scores and measurements.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, NumPy, Plotly and scikit-learn utilities. 2. Load Iris, rename the four feature columns and map target IDs to species. 3. Inspect sample rows, dataset shape, species counts and descriptive statistics. 4. Standardize the four numeric columns with StandardScaler; verify zero means and unit population standard deviations. 5. Apply PCA(n_components=2, random_state=42) to the standardized matrix. 6. Create a DataFrame of PC1 and PC2 scores, species and original measurements. 7. Inspect explained-variance percentages and principal-axis coefficients (pca.components_.T). 8. Plot PC1 versus PC2 using Plotly, with species colours, symbols and hover data. 9. Fit a four-component PCA to prepare the supporting variance bar chart. 10. Calculate species centroids and standard deviations in principal-component space. 11. Save the interactive scatter plot, static images and PCA CSV; print the analysis summary. Flow: START → Load Iris → Standardize → PCA → 2D scatter / variance chart → Save results → STOP
Program	Complete notebook code (formatting only adjusted for the page). Run in Jupyter with pandas, numpy, scikit-learn and plotly; static PNG export additionally requires a compatible Kaleido installation. <pre># Step 1: Import the required libraries import pandas as pd import numpy as np import plotly.express as px import plotly.io as pio from sklearn.datasets import load_iris from sklearn.preprocessing import StandardScaler from sklearn.decomposition import PCA import warnings warnings.filterwarnings("ignore") # Plotly renderer: interactive figure inside Jupyter + a static PNG # copy of every chart # (the PNG is what GitHub / PDF previews show, because they cannot run # interactive Plotly) try: import kaleido # noqa: F401 (only needed for the static PNG copies) pio.renderers.default = "plotly_mimetype+png" pio.renderers["png"].width = 900 pio.renderers["png"].height = 550 pio.renderers["png"].scale = 2 except ImportError: pio.renderers.default = "plotly_mimetype+notebook" # Step 2: Load the Iris dataset into a pandas DataFrame iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "sepal length (cm)": "Sepal Length (cm)", "sepal width (cm)": "Sepal Width (cm)", "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)", }) df["Species"] = df["target"].map(dict(enumerate(iris.target_names)))</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
df = df.drop(columns="target")
feature_columns = [
    "Sepal Length (cm)",
    "Sepal Width (cm)",
    "Petal Length (cm)",
    "Petal Width (cm)",
]
df.head(10)
# Step 3: Basic verification of the dataset
print("Dataset Shape:", df.shape)
print("\nSpecies Counts:")
print(df["Species"].value_counts())
print("\nDescriptive Statistics:")
print(df[feature_columns].describe().round(2))
# Step 4: Standardize the four features (PCA is sensitive to feature
# scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_columns])
scaled_df = pd.DataFrame(X_scaled, columns=feature_columns)
print("Mean of each standardized feature (should be ~0):")
print(scaled_df.mean().round(3))
print(
    "\nStandard deviation of each standardized feature (should be ~1):"
)
print(scaled_df.std(ddof=0).round(3))
# Step 5: Apply PCA and keep the first two principal components
pca = PCA(n_components=2, random_state=42)
principal_components = pca.fit_transform(X_scaled)
pca_df = pd.DataFrame(
    principal_components, columns=["PC1", "PC2"]
)
pca_df["Species"] = df["Species"]
# keep the original measurements next to the components (used for
# hover information)
pca_df = pd.concat([pca_df, df[feature_columns]], axis=1)
pca_df.head(10)
# Step 6: Explained variance and component loadings
explained = pca.explained_variance_ratio_ * 100
print("Explained Variance Ratio:")
print("PC1 :", round(explained[0], 2), "%")
print("PC2 :", round(explained[1], 2), "%")
print(
    "Total variance captured by PC1 + PC2 :",
    round(explained.sum(), 2),
    "%",
)
loadings = pd.DataFrame(
    pca.components_.T,
    columns=["PC1", "PC2"],
    index=feature_columns,
)
print("\nComponent Loadings (contribution of each feature):")
print(loadings.round(3))
# Step 7: Interactive 2D scatter plot of PC1 vs PC2 coloured by
# species
fig = px.scatter(
    pca_df,
    x="PC1",
    y="PC2",
    color="Species",
    symbol="Species",
    hover_data={
        "PC1": ":.3f",
        "PC2": ":.3f",
        "Sepal Length (cm)": True,
        "Sepal Width (cm)": True,
        "Petal Length (cm)": True,
        "Petal Width (cm)": True,
    },
),
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
title="PCA of the Iris Dataset: First Two Principal Components",
labels={
    "PC1": f"PC1 ({explained[0]:.2f}% variance)",
    "PC2": f"PC2 ({explained[1]:.2f}% variance)",
},
)
fig.update_traces(
    marker=dict(
        size=10, opacity=0.85, line=dict(width=1, color="white")
    )
)
fig.update_layout(
    legend_title_text="Species",
    width=900,
    height=550,
    template="plotly_white",
)
fig.show()
# Step 8: Scree plot - variance explained by every component
# (supporting chart)
pca_full = PCA(n_components=4, random_state=42).fit(X_scaled)
scree_df = pd.DataFrame(
    {
        "Principal Component": ["PC1", "PC2", "PC3", "PC4"],
        "Explained Variance (%)": (
            pca_full.explained_variance_ratio_ * 100
        ).round(2),
    }
)
scree_df["Cumulative Variance (%)"] = (
    scree_df["Explained Variance (%)"].cumsum().round(2)
)
print(scree_df)
fig_scree = px.bar(
    scree_df,
    x="Principal Component",
    y="Explained Variance (%)",
    text="Explained Variance (%)",
    title="Scree Plot: Variance Explained by Each Principal Component",
)
fig_scree.update_traces(textposition="outside")
fig_scree.update_layout(
    width=900, height=550, template="plotly_white"
)
fig_scree.show()
# Step 9: Analyse the clusters in the principal-component space
centroids = (
    pca_df.groupby("Species")[["PC1", "PC2"]].mean().round(3)
)
print("Centroid of each species in PC space:")
print(centroids)
spread = pca_df.groupby("Species")[["PC1", "PC2"]].std().round(3)
print(
    "\nSpread (standard deviation) of each species in PC space:"
)
print(spread)
# Step 10: Save the interactive chart, a static image and the PCA
# dataset
fig.write_html("program16_pca_scatter.html")
print(
    "Interactive PCA scatter plot saved as program16_pca_scatter.html"
)
try:
    fig.write_image(
        "program16_pca_scatter.png",
        width=900,
        height=550,
        scale=2,
    )
)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
fig_scree.write_image(  
    "program16_scree_plot.png", width=900, height=550, scale=2  
)  
print(  
    "Static images saved as program16_pca_scatter.png and program16_scree_plot.png"  
)  
except Exception as error:  
    print(  
        "Static image not saved (kaleido not available):", error  
    )  
pca_df.to_csv("program16_iris_pca.csv", index=False)  
print("PCA dataset saved as program16_iris_pca.csv")  
# Step 11: Summary of the analysis  
print("PCA ANALYSIS SUMMARY")  
print("-" * 45)  
print("Total samples           :", len(pca_df))  
print("Original features         :", len(feature_columns))  
print("Components retained       : 2")  
print("Variance explained by PC1  :", round(explained[0], 2), "%")  
print("Variance explained by PC2  :", round(explained[1], 2), "%")  
print(  
    "Total variance retained   :", round(explained.sum(), 2), "%"  
)  
print(  
    "Most influential feature  :",  
    loadings["PC1"].abs().idxmax(),  
    "(on PC1)",  
)  
print("Well separated species    : setosa")  
print("Partially overlapping     : versicolor and virginica")
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

**Output
Snapshots
With
Explanation/
Observations**

Outputs below were regenerated by executing the supplied notebook code. The document contains static snapshots; interactive hover, zoom and (for 3D) rotation are available in the HTML produced by the notebook. Notebook display calls were suppressed only during headless verification.

Dataset Shape: (150, 5)

Species Counts:

```
Species
setosa      50
versicolor 50
virginica   50
```

Name: count, dtype: int64

Descriptive Statistics:

	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)
count	150.00	150.00	150.00	150.00
mean	5.84	3.06	3.76	1.20
std	0.83	0.44	1.77	0.76
min	4.30	2.00	1.00	0.10
25%	5.10	2.80	1.60	0.30
50%	5.80	3.00	4.35	1.30
75%	6.40	3.30	5.10	1.80
max	7.90	4.40	6.90	2.50

Mean of each standardized feature (should be ~0):

```
Sepal Length (cm)  -0.0
Sepal Width (cm)   -0.0
Petal Length (cm)  -0.0
Petal Width (cm)   -0.0
dtype: float64
```

Standard deviation of each standardized feature (should be ~1):

```
Sepal Length (cm)  1.0
Sepal Width (cm)   1.0
Petal Length (cm)  1.0
Petal Width (cm)   1.0
dtype: float64
```

Fig. 1.1: Dataset verification and feature standardization

The dataset contains 150 records and five DataFrame columns: four measurements and one species label. Each species has 50 observations. The transformed feature matrix has zero means (up to floating-point rounding) and population standard deviations of one.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Explained Variance Ratio:

PC1 : 72.96 %

PC2 : 22.85 %

Total variance captured by PC1 + PC2 : 95.81 %

Component Loadings (contribution of each feature):

	PC1	PC2
Sepal Length (cm)	0.521	0.377
Sepal Width (cm)	-0.269	0.923
Petal Length (cm)	0.580	0.024
Petal Width (cm)	0.565	0.067

Fig. 1.2: Explained variance and principal-axis coefficients

PC1 and PC2 retain 95.81% of the variance. Petal length has the largest absolute PC1 coefficient (0.580). The notebook calls these values loadings; here they are the principal-axis coefficients from `pca.components_.T`, not correlation loadings.

PCA of the Iris Dataset: First Two Principal Components

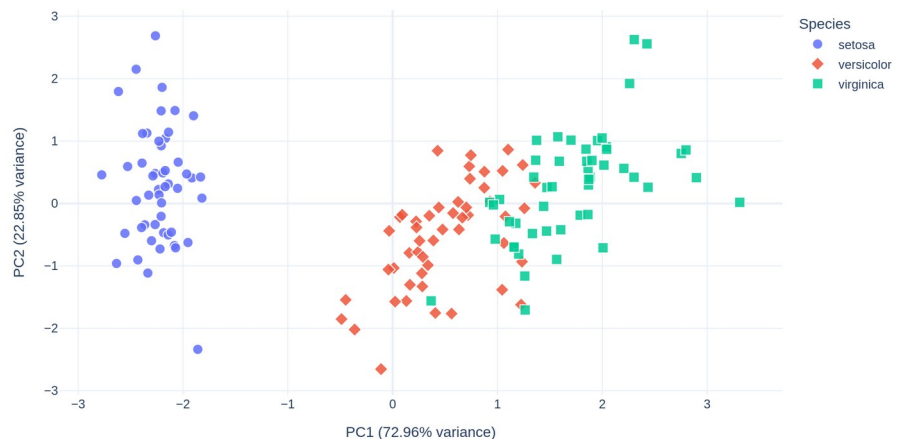


Fig. 1.3: Plotly scatter plot of the first two principal components

Setosa occupies the negative PC1 region and is separated from the other species. Versicolor and virginica overlap partly. Colours and symbols represent known species, not clusters generated by PCA.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Scree Plot: Variance Explained by Each Principal Component

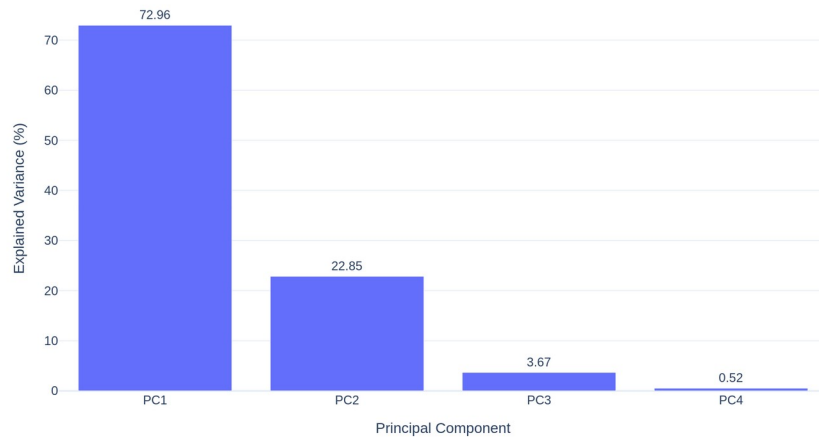


Fig. 1.4: Variance explained by all four principal components

The four variance percentages are 72.96%, 22.85%, 3.67% and 0.52%. Retaining the first two components loses approximately 4.19% of the standardized variance.

Centroid of each species in PC space:

	PC1	PC2
Species		
setosa	-2.225	0.289
versicolor	0.496	-0.550
virginica	1.728	0.261

Spread (standard deviation) of each species in PC space:

	PC1	PC2
Species		
setosa	0.206	0.934
versicolor	0.464	0.833
virginica	0.571	0.876

Fig. 1.5: Species centroids and spread in principal-component space

Mean PC1 scores increase from setosa (-2.225) through versicolor (0.496) to virginica (1.728). Species labels are used only after projection for these summaries. The reported spreads are sample standard deviations.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

	Interactive PCA scatter plot saved as program16_pca_scatter.html Static images saved as program16_pca_scatter.png and program16_scree_plot.png PCA dataset saved as program16_iris_pca.csv PCA ANALYSIS SUMMARY <pre>----- Total samples : 150 Original features : 4 Components retained : 2 Variance explained by PC1 : 72.96 % Variance explained by PC2 : 22.85 % Total variance retained : 95.81 % Most influential feature : Petal Length (cm) (on PC1) Well separated species : setosa Partially overlapping : versicolor and virginica</pre> Fig. 1.6: File export confirmation and PCA summary Execution produced the interactive scatter HTML, two static charts and a CSV containing PC1, PC2, species and the four original measurements. The summary confirms 150 observations and 95.81% retained variance.
Conclusions	The Iris dataset was successfully standardized and reduced from four measurements to two principal components. PC1 explains 72.96% and PC2 explains 22.85% of the standardized variance, retaining 95.81% in total. Setosa is clearly separated in the projection, while versicolor and virginica partly overlap. PCA is an unsupervised dimensionality-reduction method, not a clustering algorithm; species colours are reference labels. Plotly provides an interactive view, and the saved HTML and CSV support further exploration.
References	1. scikit-learn PCA documentation: 2. scikit-learn Iris dataset / StandardScaler documentation: 3. Plotly Python documentation: 4. Pandas documentation: 5. Python notebook (GitHub link): 6. Class lecture link: 7. Specimen copy link: 8. Template link:

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Title	Visualizing Clusters with scikit-learn and Plotly			
Problem Statement	Perform K-means clustering on the Iris dataset and visualize the clusters using an interactive 3D scatter plot in Plotly.			
Objectives	• Prepare and standardize the four Iris measurements. • Compare inertia and silhouette scores for candidate cluster counts. • Fit K-means with three clusters and inspect their composition. • Evaluate clusters using silhouette score and adjusted Rand index. • Visualize clusters and their centres in an interactive Plotly 3D scatter plot.			
Dataset URL Link / File	Built-in Iris dataset: sklearn.datasets.load_iris(as_frame=True). 150 samples; four numeric features measured in cm: sepal length, sepal width, petal length and petal width. Species: setosa (50), versicolor (50), virginica (50). Species labels are used for visualization/evaluation, not as input features. Dataset URL link:			
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads the Iris dataset and labels.
	2	StandardScaler()	sklearn	Standardizes all four numeric features.
	3	KMeans()	sklearn	Defines the K-means clustering model.
	4	fit_predict()	sklearn	Fits the model and returns cluster labels.
	5	silhouette_score()	sklearn	Measures cluster cohesion and separation.
	6	adjusted_rand_score()	sklearn	Compares clusters with known species labels.
	7	crosstab()	Pandas	Tabulates cluster and species membership.
	8	inverse_transform()	sklearn	Converts centres back to original cm units.
	9	line()	Plotly	Plots inertia for the candidate k values.
	10	scatter_3d()/Scatter3d()	Plotly	Plots observations and cluster centres in 3D.
	11	write_html()/write_image()	Plotly	Saves interactive HTML and static charts.
	12	to_csv()	Pandas	Saves original features and cluster labels.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, NumPy, Plotly and the scikit-learn clustering/evaluation utilities. 2. Load Iris, rename the feature columns and map the known species labels. 3. Standardize only the four measurements using StandardScaler. 4. Fit K-means for $k = 1$ to 10; collect inertia and silhouette scores (silhouette is undefined for $k = 1$). 5. Plot inertia versus k. Use $k = 3$ to compare with the three known species; do not claim it maximizes the silhouette score. 6. Fit KMeans($n_clusters=3$, $init="k-means++"$, $n_init=10$, $random_state=42$) and assign cluster labels. 7. Compute cluster sizes, a cluster/species cross-tabulation, silhouette score and adjusted Rand index. 8. Convert the cluster centres from standardized units back to cm. 9. Plot sepal length, sepal width and petal length in 3D; use colour for cluster, symbol for species and hover data for petal width. 10. Overlay cluster centres, save the interactive HTML, static charts and clustered CSV, and print the summary. Flow: START → Load Iris → Standardize → Compare k → Fit K-means → Evaluate → 3D scatter → Save results → STOP
Program	Complete notebook code (formatting only adjusted for the page). Run in Jupyter with pandas, numpy, scikit-learn and plotly; static PNG export additionally requires a compatible Kaleido installation. <pre># Step 1: Import the required libraries import pandas as pd import numpy as np import plotly.express as px import plotly.graph_objects as go import plotly.io as pio from sklearn.datasets import load_iris from sklearn.preprocessing import StandardScaler from sklearn.cluster import KMeans from sklearn.metrics import silhouette_score, adjusted_rand_score import warnings warnings.filterwarnings("ignore") # Plotly renderer: interactive figure inside Jupyter + a static PNG # copy of every chart # (the PNG is what GitHub / PDF previews show, because they cannot run # interactive Plotly) try: import kaleido # noqa: F401 (only needed for the static PNG copies) pio.renderers.default = "plotly_mimetype+png" pio.renderers["png"].width = 900 pio.renderers["png"].height = 550 pio.renderers["png"].scale = 2 except ImportError: pio.renderers.default = "plotly_mimetype+notebook" # Step 2: Load the Iris dataset into a pandas DataFrame iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "sepal length (cm)": "Sepal Length (cm)", "sepal width (cm)": "Sepal Width (cm)", "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)", }) df["Species"] = df["target"].map(</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
dict(enumerate(iris.target_names))
)
df = df.drop(columns="target")
feature_columns = [
    "Sepal Length (cm)",
    "Sepal Width (cm)",
    "Petal Length (cm)",
    "Petal Width (cm)",
]
print("Dataset Shape:", df.shape)
print("\nSpecies Counts:")
print(df["Species"].value_counts())
df.head(10)
# Step 3: Standardize the features so that every measurement has equal
# weight in the distance calculation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_columns])
print("Scaled feature matrix shape:", X_scaled.shape)
print("First 5 scaled rows:")
print(np.round(X_scaled[:5], 3))
# Step 4: Elbow method and silhouette score to choose the number of
# clusters (k)
k_values = range(1, 11)
inertia = []
silhouette = []
for k in k_values:
    model = KMeans(
        n_clusters=k, init="k-means++", n_init=10, random_state=42
    )
    labels = model.fit_predict(X_scaled)
    inertia.append(model.inertia_)
    silhouette.append(
        silhouette_score(X_scaled, labels) if k > 1 else np.nan
    )
elbow_df = pd.DataFrame(
    {
        "k": list(k_values),
        "Inertia (WCSS)": np.round(inertia, 3),
        "Silhouette Score": np.round(silhouette, 3),
    }
)
print(elbow_df.to_string(index=False))
fig_elbow = px.line(
    elbow_df,
    x="k",
    y="Inertia (WCSS)",
    markers=True,
    title="Elbow Method: Inertia vs Number of Clusters (k)",
)
fig_elbow.update_layout(
    width=900,
    height=550,
    template="plotly_white",
    xaxis=dict(dtick=1),
)
fig_elbow.show()
# Step 5: Fit K-means with k = 3 (three Iris species) and assign
# cluster labels
kmeans = KMeans(
    n_clusters=3, init="k-means++", n_init=10, random_state=42
)
df["Cluster"] = kmeans.fit_predict(X_scaled)
df["Cluster"] = "Cluster " + df["Cluster"].astype(str)
print("Number of iterations to converge:", kmeans.n_iter_)
print("Final inertia (WCSS):", round(kmeans.inertia_, 3))
print("\nCluster Sizes:")
print(df["Cluster"].value_counts().sort_index())
df.head(10)
# Step 6: Compare the discovered clusters with the actual species
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
comparison = pd.crosstab(df["Cluster"], df["Species"])
print("Cluster vs Species Cross-tabulation:")
print(comparison)
sil = silhouette_score(X_scaled, kmeans.labels_)
ari = adjusted_rand_score(iris.target, kmeans.labels_)
print("\nSilhouette Score (k = 3) : ", round(sil, 3))
print("Adjusted Rand Index vs species:", round(ari, 3))
# percentage of points whose cluster matches the dominant species of
# that cluster
correctly_grouped = comparison.max(axis=1).sum()
print(
    "Points grouped with their dominant species:",
    correctly_grouped,
    "of",
    len(df),
    "(",
    round(correctly_grouped / len(df) * 100, 2),
    "% )",
)
# Step 7: Cluster centres converted back to the original units (cm)
centers_cm = scaler.inverse_transform(kmeans.cluster_centers_)
centers_df = pd.DataFrame(
    centers_cm, columns=feature_columns
).round(3)
centers_df.insert(
    0, "Cluster", ["Cluster 0", "Cluster 1", "Cluster 2"]
)
print("Cluster Centres (original units):")
print(centers_df.to_string(index=False))
# Step 8: Interactive 3D scatter plot of the clusters (with cluster
# centres marked)
fig = px.scatter_3d(
    df.sort_values("Cluster"),
    x="Sepal Length (cm)",
    y="Sepal Width (cm)",
    z="Petal Length (cm)",
    color="Cluster",
    symbol="Species",
    hover_data={"Petal Width (cm)": True},
    title="K-means Clustering of the Iris Dataset (k = 3): Interactive 3D View",
    opacity=0.8,
)
fig.update_traces(marker=dict(size=4))
# add the three cluster centres as large black diamonds
fig.add_trace(
    go.Scatter3d(
        x=centers_df["Sepal Length (cm)"],
        y=centers_df["Sepal Width (cm)"],
        z=centers_df["Petal Length (cm)"],
        mode="markers",
        marker=dict(size=8, color="black", symbol="diamond"),
        hovertext=centers_df["Cluster"] + " centre",
        name="Cluster Centres",
    )
)
fig.update_layout(
    width=900,
    height=650,
    legend_title_text="Cluster / Species",
    margin=dict(l=0, r=0, b=0, t=60),
    scene=dict(
        xaxis_title="Sepal Length (cm)",
        yaxis_title="Sepal Width (cm)",
        zaxis_title="Petal Length (cm)",
        aspectmode="cube",
        camera=dict(eye=dict(x=1.7, y=1.5, z=0.9)),
    ),
    template="plotly_white",
)
fig.show()
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
# Step 9: Save the interactive chart, static images and the clustered
# dataset
fig.write_html("program17_kmeans_3d_scatter.html")
print(
    "Interactive 3D scatter plot saved as program17_kmeans_3d_scatter.html"
)
try:
    fig.write_image(
        "program17_kmeans_3d_scatter.png",
        width=900,
        height=650,
        scale=2,
    )
    fig_elbow.write_image(
        "program17_elbow_plot.png", width=900, height=550, scale=2
    )
    print(
        "Static images saved as program17_kmeans_3d_scatter.png and program17_elbow_plot.png"
    )
except Exception as error:
    print(
        "Static image not saved (kaleido not available):", error
    )
df.to_csv("program17_iris_kmeans_clusters.csv", index=False)
print(
    "Clustered dataset saved as program17_iris_kmeans_clusters.csv"
)
# Step 10: Summary of the analysis
print("K-MEANS CLUSTERING SUMMARY")
print("-" * 45)
print("Total samples           :", len(df))
print(
    "Features used             :",
    len(feature_columns),
    "(standardized)",
)
print("Number of clusters (k)    : 3")
print(
    "Cluster sizes             :",
    df["Cluster"].value_counts().sort_index().tolist(),
)
print("Silhouette score          :", round(sil, 3))
print("Adjusted Rand Index       :", round(ari, 3))
print(
    "Agreement with species    :",
    round(correctly_grouped / len(df) * 100, 2),
    "%",
)
print("Perfectly separated species : setosa")
print("Overlapping species        : versicolor and virginica")
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Output Snapshots With Explanation/ Observations	Outputs below were regenerated by executing the supplied notebook code. The document contains static snapshots; interactive hover, zoom and (for 3D) rotation are available in the HTML produced by the notebook. Notebook display calls were suppressed only during headless verification.
	<pre>Dataset Shape: (150, 5) Species Counts: Species setosa 50 versicolor 50 virginica 50 Name: count, dtype: int64 Scaled feature matrix shape: (150, 4) First 5 scaled rows: [[-0.901 1.019 -1.34 -1.315] [-1.143 -0.132 -1.34 -1.315] [-1.385 0.328 -1.397 -1.315] [-1.507 0.098 -1.283 -1.315] [-1.022 1.249 -1.34 -1.315]]</pre> Fig. 2.1: Dataset and standardized feature matrix All 150 observations and four numeric features are used for clustering. Species labels are kept separately for plotting and evaluation; they are not included in the distance calculations.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

k	Inertia (WCSS)	Silhouette Score
1	600.000	NaN
2	222.362	0.582
3	139.820	0.460
4	114.093	0.387
5	90.928	0.346
6	81.544	0.317
7	72.631	0.320
8	62.541	0.339
9	55.119	0.342
10	47.391	0.352

Fig. 2.2: Inertia and silhouette scores for k = 1 to 10

Inertia decreases as more clusters are added. The highest silhouette score among the tested values is at k = 2 (0.582), not k = 3 (0.460). The notebook uses k = 3 for comparison with the three known Iris species. A silhouette score is undefined for a single cluster.

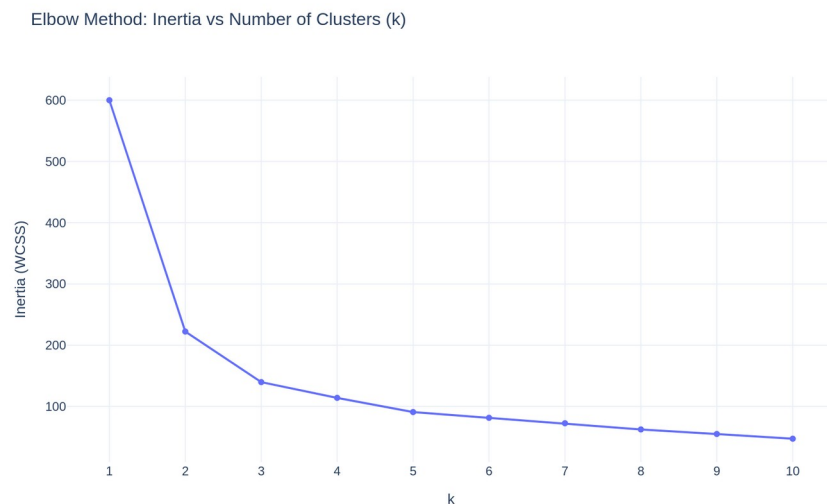


Fig. 2.3: Plotly elbow chart

The large early reductions in inertia are followed by progressively smaller improvements. This chart supports exploring a small number of clusters but does not prove that three is uniquely optimal.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
Number of iterations to converge: 4
Final inertia (WCSS): 139.82
```

Cluster Sizes:

Cluster

Cluster 0 53

Cluster 1 50

Cluster 2 47

Name: count, dtype: int64

Cluster vs Species Cross-tabulation:

Species	setosa	versicolor	virginica
---------	--------	------------	-----------

Cluster

Cluster 0	0	39	14
-----------	---	----	----

Cluster 1	50	0	0
-----------	----	---	---

Cluster 2	0	11	36
-----------	---	----	----

Silhouette Score (k = 3) : 0.46

Adjusted Rand Index vs species: 0.62

Points grouped with their dominant species: 125 of 150 (83.33 %)

Fig. 2.4: Cluster sizes, species cross-tabulation and evaluation

Cluster 1 contains all 50 setosa samples. Cluster 0 contains 39 versicolor and 14 virginica; Cluster 2 contains 11 versicolor and 36 virginica. Cluster IDs are arbitrary. The dominant-species counts are 39, 50 and 36 for Clusters 0, 1 and 2, respectively: 125/150 (83.33%) in total.

Cluster Centres (original units):

Cluster	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)
---------	-------------------	------------------	-------------------	------------------

Cluster 0	5.802	2.674	4.370	1.413
-----------	-------	-------	-------	-------

Cluster 1	5.006	3.428	1.462	0.246
-----------	-------	-------	-------	-------

Cluster 2	6.781	3.096	5.511	1.972
-----------	-------	-------	-------	-------

Fig. 2.5: Cluster centres in centimetres

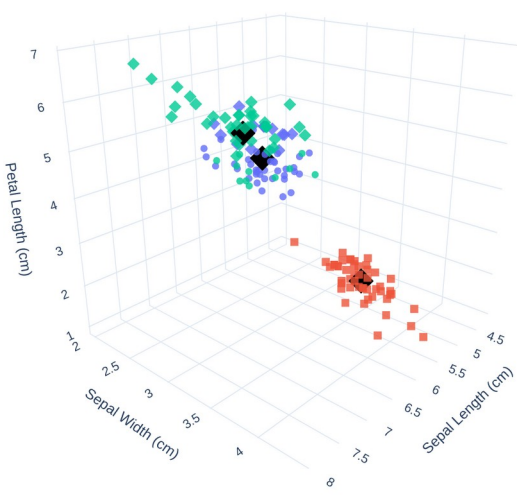
Inverse transformation makes each centre interpretable in original measurement units. All four dimensions contribute to model fitting, even though only three dimensions can be displayed in the scatter plot.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

	K-means Clustering of the Iris Dataset (k = 3): Interactive 3D View  Fig. 2.6: Interactive 3D cluster plot with centres The static view shows sepal length, sepal width and petal length. Colour encodes cluster and symbol encodes species. Black diamonds mark cluster centres. Petal width is available on hover in the interactive HTML.
	Interactive 3D scatter plot saved as program17_kmeans_3d_scatter.html Static images saved as program17_kmeans_3d_scatter.png and program17_elbow_plot.png Clustered dataset saved as program17_iris_kmeans_clusters.csv K-MEANS CLUSTERING SUMMARY <pre> ----- Total samples : 150 Features used : 4 (standardized) Number of clusters (k) : 3 Cluster sizes : [53, 50, 47] Silhouette score : 0.46 Adjusted Rand Index : 0.62 Agreement with species : 83.33 % Perfectly separated species : setosa Overlapping species : versicolor and virginica </pre> Fig. 2.7: File export confirmation and K-means summary Execution converged in four iterations with inertia 139.820. It saved the interactive 3D scatter, two static charts and the clustered dataset. The output confirms cluster sizes 53, 50 and 47, silhouette 0.460 and adjusted Rand index 0.620.
Conclusions	K-means successfully grouped 150 standardized Iris observations into three clusters of sizes 53, 50 and 47. The final inertia is 139.820, the silhouette score is 0.460 and the adjusted Rand index is 0.620. All 50 setosa samples form one cluster; versicolor and virginica overlap. Dominant-species agreement is 125/150 (83.33%), a descriptive cluster-purity measure rather than supervised test accuracy. The 3D Plotly chart displays three original measurements, while clustering uses all four standardized features. The result demonstrates useful but imperfect agreement between unsupervised groups and biological species.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

References

1. scikit-learn K-means documentation:
2. scikit-learn Iris dataset / StandardScaler documentation:
3. Plotly Python documentation:
4. Pandas documentation:
5. Python notebook (GitHub link):
6. Class lecture link:
7. Specimen copy link:
8. Template link: